

Lab 3 — Component Modelling & Architectural Pattern Selection

Group Travel Expense Simplification Engine

Problem Statement #37

Archana Shivakumar
PES1UG24CS078

Architecture Selection

We chose a Microservices Architecture for the Group Travel Expense Simplification Engine system. The system decomposes naturally into independent bounded contexts — trip/membership management, expense recording, currency conversion, and debt-settlement computation — that have different scaling, reliability, and change profiles, which microservices is best suited to address.

Two Reasons for This Choice

- **Independent scaling of the settlement computation.** FR-001 and NFR-001 require the min-flow debt-settlement algorithm to complete in under 100 ms for up to 50 travelers and 500 expenses, and many travellers tend to open the settlement view at the same time, right at the end of a trip. Packaging the Settlement Engine as its own service lets it be scaled and resourced independently of lighter services such as Trip & Membership Management, instead of over-provisioning the whole application to absorb that spike.
- **Fault isolation around the external currency dependency.** FR-003 and NFR-002 require graceful degradation when the external Currency Exchange API is slow or unreachable — falling back to the last cached rate and flagging affected expenses. Isolating this in its own Currency Conversion Service means an outage in that one dependency cannot cascade into trip creation, expense logging, or settlement viewing, which keep working normally.

Security Advantage

Each microservice enforces its own access boundary and exposes only the minimal interface it needs. Only the Currency Conversion Service ever communicates with the external Currency Exchange API — the Settlement Engine and the trip/expense data are never directly reachable from that third-party integration. This limits the blast radius if the external API integration were ever compromised or spoofed, and lets sensitive operations (membership changes, balance data) be authorized and audited independently of the external currency dependency.

Performance Benefit

Because the Settlement Engine is a separate, independently deployable service, it can run with dedicated compute and an in-memory view of the current trip's ledger, so the min-flow computation is not competing with request traffic to the other services. This directly supports NFR-001's under-100 ms target even when several Group Travelers request the settlement view concurrently at the end of a trip, since Settlement Engine instances can be scaled horizontally behind that load without being blocked by Trip Management or Report Export traffic.

